

# 于鸿伟\_九齿skill创新挑战\_中期报告

赛事：2026 春季启元人工智能大赛  
赛道：九齿 .skill 创新挑战  
报告类型：初赛中期报告  
小组名称：于鸿伟（个人参赛）  
GitHub ID：hongwei-2026  
赛题编号：T3-1-1  
仓库：[github.com/hongwei-2026/qiyuanbisai](https://github.com/hongwei-2026/qiyuanbisai)  
报告日期：2026 年 6 月 8 日

## 摘要

本作品交付 ntops-forge (v1.0, 主作品) 与 ntops-copilot (v0.5, 轻量备选) 双 Skill 套件，核心创新是将 Agent 写 ntops 算子的过程从「散文化手册」升级为 PLAN→CODEGEN→GUARD→PROVE→SHIP 五段可执行工厂流水线，配套 16+ 脚本、12 张 GPU 实测截图、jsonl 审计与 A/B 量化证据。

在 RTX 4080 环境，forge gate 一键验收 五算子 (silu / add / gelu / relu / mul)：每算子 pytest 8/8 通过、单算子约 7 秒。run\_baseline\_demo.py 提供可复现无 Skill 基线证据；相对 Baseline, preflight 0%→100%，步骤 6→1，人工介入 4→0。

关键词：算子工厂、可执行 Skill、语义对照、A/B 量化、GPU 实测

## 一、.skill 名称、赛题与小组信息

| 项目       | 内容                              |
|----------|---------------------------------|
| 主 Skill  | ntops-forge（九齿算子工厂）             |
| 辅助 Skill | ntops-copilot（轻量副驾驶）            |
| 赛题编号     | T3-1-1（暂定，以赛题组通知为准）             |
| 小组名称     | 于鸿伟                             |
| 建议 PR 分支 | 2026-spring-hongwei-2026-T3-1-1 |
| 建议 PR 标题 | [2026春季][T3-1-1] hongwei-2026   |

初赛以独立 skill 仓库 + commit + zip 提交；最终提交载体以后续赛题组通知为准。

## 二、核心创新点总览（本作品 vs 文档型 Skill）

| 编号 | 创新点            | 解决的问题             | 可验证证据                         |
|----|----------------|-------------------|-------------------------------|
| C1 | 五段工厂流水线        | Agent 命令零散、易漏步骤   | forge gate 一键跑通五算子            |
| C2 | 规格驱动公式注入       | 手写 application 易错 | spec formula 自动写入内核           |
| C3 | 语义对照顾栏         | 公式写对但语义不一致        | compare_ref matches reference |
| C4 | 结构护栏 preflight | Triton 稿混入交卷      | Triton 5 项 FAIL / forge OK    |
| C5 | 失败诊断 fix_cards | 报错后不知怎修           | FC-001/004/012 自动匹配           |
| C6 | jsonl 全链路审计    | 无法复现 Agent 过程     | 每阶段 ok+耗时可追溯                  |
| C7 | 自然语言生成 spec    | 新算子规格起草慢          | NL→YAML spec 秒级生成             |
| C8 | 五算子 gate 回归    | 覆盖算子少、难服评委        | silu/add/gelu/relu/mul 一键验收   |

与文档型 Skill 的本质差异：同类作品多只有 SKILL.md；本作品把规范落到可运行脚本链，失败在秒级本地/GPU 前移到 GUARD，而非推到 CI 才暴露。

### 三、.skill 目标、设计原则与包结构

#### 3.1 要解决的三大类 Agent 失败

1. 范式错误：生成 Triton @triton.jit，不符合 ntops premake 评审要求
2. 流程断裂：漏 \_\_init\_\_.py 注册、漏 torch 封装、漏 pytest、漏 PR 规范
3. 无法自检：结构/语义错误推到 CI/GPU 才暴露，浪费机时

#### 3.2 设计原则

- ☐ 可执行优先：每个阶段对应脚本，不只写文档
- ☐ 五段闭环：PLAN → CODEGEN → GUARD → PROVE → SHIP
- ☐ 精准测试：只跑 tests/test\_<op>.py，禁止全量 pytest
- ☐ 可审计：forge\_runs.jsonl 记录每阶段结果与耗时

#### 3.3 包结构

| 路径                    | 内容                                          |
|-----------------------|---------------------------------------------|
| skills/ntops-forge/   | SKILL.md, specs/, taxonomy.md, fix_cards.md |
| skills/ntops-copilot/ | SKILL.md, tasks/, formulas.md               |
| scripts/              | forge.py 等 16 个可执行脚本                        |

| 路径                | 内容                              |
|-------------------|---------------------------------|
| docs/screenshots/ | 16 张 GPU 实测截图（本报告附图，含最终 5v5 验收） |



| 脚本                         | 功能             |
|----------------------------|----------------|
| doctor.py                  | GPU/依赖环境自检     |
| forge_diagnose.py          | 失败文本→fix_cards |
| forge_spec.py              | 自然语言→YAML spec |
| record_run.py / eval_ab.py | A/B 数据记录与报告    |

```
(base) root@autodl-container-6764489861-420726ac:~/work/skill# source /root/miniconda3/bin/activate base
cd /root/work/skill
mkdir -p docs/screenshots docs/demo-logs
(base) root@autodl-container-6764489861-420726ac:~/work/skill# nvidia-smi -L
python scripts/doctor.py | tee docs/demo-logs/00-doctor.log
GPU 0: NVIDIA GeForce RTX 4080 (UUID: GPU-d859f103-f602-9b9a-2491-74c92c28cc9b)
ntops-copilot doctor
-----
OK python executable found
OK ninetoothed importable
OK torch importable
OK pytest importable
OK CUDA available: NVIDIA GeForce RTX 4080
OK ntops installed
OK nvidia-smi: GPU 0: NVIDIA GeForce RTX 4080 (UUID: GPU-d859f103-f602-9b9a-2491-74c92c28cc9b)
OK git available
-----
doctor finished
(base) root@autodl-container-6764489861-420726ac:~/work/skill#
```

图2 环境自检 doctor

图2 doctor 确认: ninetoothed / torch / pytest / ntops / RTX 4080 CUDA 就绪

```
OK: prove passed
== [SHIP] ==
PR branch: 2026-spring-hongwei-2026-T1-1-1
PR title: [2026-spring-hongwei-2026-T1-1-1] hongwei-2026
Kernel: /tmp/gelu_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task gelu --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment.gelu -> /root/work/skill/docs/ab_runs.csv

OK: ship pack ready

== FORGE OK (gelu) in 6.8s ==
Audit log: /root/work/skill/docs/forge_runs.jsonl

== GATE SUMMARY ==
OK silu
OK add
OK gelu
GATE OK: all operators passed
WARN: /root/work/ntops/src/ntops/kernels/silu.py exists; using /tmp (pass --force to overwrite)

== [PLAN] ==
op=silu family=elementwise_unary pattern=unary
pytest=tests/test_silu.py reference=src/ntops/kernels/silu.py
OK: plan ready

== [CODEGEN] ==
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_kernel.py --name silu --pattern unary --style ntops --out /tmp/silu_forge_kernel.py --formula output = input / (1 + ntl.exp(-ntl.cast(input, ntl.float32)))
Wrote /tmp/silu_forge_kernel.py [ntops/unary] *formula
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_torch.py --name silu --pattern unary --out /tmp/silu_forge_torch.py
Wrote /tmp/silu_forge_torch.py

OK: codegen finished

== [GUARD] ==
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_forge_kernel.py --kernel --strict
OK: preflight passed
$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/silu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/silu.py
OK: application() matches reference

OK: guard passed

== [PROVE] ==
$ /root/miniconda3/bin/python -m pytest -q tests/test_silu.py
.....
8 passed in 3.56s

OK: prove passed

== [SHIP] ==
PR branch: 2026-spring-hongwei-2026-T1-1-1
PR title: [2026-spring-hongwei-2026-T1-1-1] hongwei-2026
Kernel: /tmp/silu_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
```

图3 单算子五段流水线

图3 forge run silu: PLAN→CODEGEN→GUARD→PROVE→SHIP 全通过, 6.8s

```

OK: application() matches reference
OK: guard passed
== [PROVE] ==
$ /root/miniconda3/bin/python -m pytest -q tests/test_gelu.py
*****
8 passed, 8 skipped in 3.52s
OK: prove passed
== [SHIP] ==
PR branch: 2026-spring-hongwei-2026-TI-1-1
PR title: [2026春季][TI-1-1] hongwei-2026
Kernel: tmp/gelu_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task gelu --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment/gelu -> /root/work/skill/docs/ab_runs.csv
OK: ship pack ready
== FORCE OK (gelu) in 6.9s ==
Audit log: /root/work/skill/docs/forge_runs.jsonl
== GATE SUMMARY ==
OK silu
OK add
OK gelu
GATE OK: all operators passed
(base) root@autodl-container-6764489861-420726ac:~/work/skill# grep -E "PLAN|GUARD|PROVE|matches reference|passed|GATE" docs/demo-logs/01-forge-gate.log
== [PLAN] ==
== [GUARD] ==
OK: preflight passed
OK: application() matches reference
OK: guard passed
== [PROVE] ==
8 passed in 3.54s
OK: prove passed
== [PLAN] ==
== [GUARD] ==
OK: preflight passed
OK: application() matches reference
OK: guard passed
== [PROVE] ==
8 passed in 3.57s
OK: prove passed
== [PLAN] ==
== [GUARD] ==
OK: preflight passed
OK: application() matches reference
OK: guard passed
== [PROVE] ==
8 passed, 8 skipped in 3.52s
OK: prove passed
== GATE SUMMARY ==
GATE OK: all operators passed
(base) root@autodl-container-6764489861-420726ac:~/work/skill#

```

图4 一键 gate 三算子验收

图4 forge gate: silu / add / gelu / relu / mul 五算子 GATE OK

```

$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task add --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment/add -> /root/work/skill/docs/ab_runs.csv
OK: ship pack ready

== FORCE OK (add) in 7.0s ==
Audit log: /root/work/skill/docs/forge_runs.jsonl
WARN: /root/work/ntops/src/ntops/kernels/gelu.py exists; using /tmp (pass --force to overwrite)

== [PLAN] ==
op-gelu family=elementwise unary pattern=unary
pytest: tests/test_gelu.py reference=src/ntops/kernels/gelu.py
OK: plan ready

== [CODEGEN] ==
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_kernel.py --name gelu --pattern unary --style ntops --out /tmp/gelu_forge_kernel.py --formula output = input * 0.5 * (1 + ntl.erf(input / ntl.sqrt(2.0)))
Wrote /tmp/gelu_forge_kernel.py [ntops/unary] *formula
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_torch.py --name gelu --pattern unary --out /tmp/gelu_forge_torch.py
Wrote /tmp/gelu_forge_torch.py
OK: codegen finished

== [GUARD] ==
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/gelu_forge_kernel.py --kernel --strict
OK: preflight passed
$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/gelu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/gelu.py
OK: application() matches reference
OK: guard passed

== [PROVE] ==
$ /root/miniconda3/bin/python -m pytest -q tests/test_gelu.py
.s.s.s.s.s.s.s.s
8 passed, 8 skipped in 3.52s
[100%]
OK: prove passed

== [SHIP] ==
PR branch: 2026-spring-hongwei-2026-T1-1-1
PR title: [2026春季][T1-1-1] hongwei-2026
Kernel: /tmp/gelu_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task gelu --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment/gelu -> /root/work/skill/docs/ab_runs.csv
OK: ship pack ready

== FORCE OK (gelu) in 6.9s ==
Audit log: /root/work/skill/docs/forge_runs.jsonl

== GATE SUMMARY ==
OK silu
OK add
OK gelu
GATE OK: all operators passed
(base) root@auto01:container-67644689861-490726pc~/work/skill#

```

图5 gelu 流水线与 gate 汇总

图5 gelu 含 default\_application 特例；五算子 gate 汇总全部通过

## 五、七大创新点实测详解（附图）

### 创新点 C2：规格驱动公式注入

spec YAML 中 formula 字段自动注入 scaffold 生成的 application()，Agent 无需手写公式行。

```

OK: guard passed
== [PROVE] ==
$ /root/miniconda3/bin/python -m pytest -q tests/test_silu.py
.....
8 passed in 3.54s
OK: prove passed
== [SHIP] ==
PR branch: 2026-spring hongwei-2026-T1-1-1
PR title: [2026春季]T1-1-1 hongwei-2026
Kernel: /tmp/silu_forge_kernel.py
HONOR REFERENCE: update before ntops upstream PR
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task silu --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment/silu -> /root/work/skill/docs/ab_runs.csv
OK: ship pack ready
== FORCE OK (silu) in 6.8s ==
Audit log: /root/work/skill/docs/forge_runs.jsonl
(base) root@autodl-container-6764489861-420726ac:/work/skill# cat skills/ntops-forge/specs/silu.yaml
echo -
head -30 /tmp/silu_forge_kernel.py
id: forge-silu
op: silu
family: elementwise_unary
pattern: unary
spec: |
  silu(x) = x * sigmoid(x)
  formula: "output = input / (1 + ntl.exp(ntl.cast(input, ntl.float32)))"
  reference: src/ntops/kernels/silu.py
  pytest: tests/test_silu.py
  contest_id: T1-1
  github_id: hongwei-2026
  acceptance:
    preflight_strict: true
    compare_ref: true
    pytest: true
  """NineToothed kernel: silu (ntops/unary scaffold)."""
import functools
import ninetoothed.language as ntl
from ninetoothed import Tensor
from ntops.kernels.element_wise import arrangement

def application(input, output):
    output = input / (1 + ntl.exp(ntl.cast(input, ntl.float32))) # noqa: F841

def premake(ndim, dtype=None, block_size=None):
    arrangement = functools.partial(arrangement, block_size=block_size)
    tensors = (Tensor(ndim, dtype=dtype), Tensor(ndim, dtype=dtype))
    return arrangement._application_tensors
(base) root@autodl-container-6764489861-420726ac:/work/skill#

```

图6 公式注入 spec 与内核一致

图6 silu spec formula 与 /tmp/silu\_forge\_kernel.py 中 application 一致

### 创新点 C3：语义对照护栏 compare\_ref

生成稿与官方 reference 逐函数比对，gelu 支持 default\_application 特例。

```

    tensors = (tensor(ndim, dtype=dtype), tensor(ndim, dtype=dtype))
    return arrangement_, application, tensors
(base) root@autodl-container-6764489861-420726ac:~/work/skill# python scripts/compare_ref.py /tmp/silu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/silu.py
python scripts/compare_ref.py /tmp/gelu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/gelu.py 2>/dev/null || \
python scripts/forge.py run gelu --ntops-root /root/work/ntops >/dev/null 2>&1 && \
python scripts/compare_ref.py /tmp/gelu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/gelu.py
OK: application() matches reference
OK: application() matches reference
OK: application() matches reference
(base) root@autodl-container-6764489861-420726ac:~/work/skill#

```

图7 compare\_ref silu/gelu 通过

图7 三行 OK: application() matches reference

## 创新点 C4：结构护栏--拦截 Triton 错误范式

无 Skill 时 Agent 常写 Triton; preflight 秒级拒绝并列出 5 项具体违规。

```

(base) root@autodl-container-6764489861-420726ac:~/work/skill# python scripts/compare_ref.py /tmp/silu_forge_kernel.py --ref /root/work/n
python scripts/compare_ref.py /tmp/gelu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/gelu.py 2>/dev/null || \
python scripts/forge.py run gelu --ntops-root /root/work/ntops >/dev/null 2>&1 && \
python scripts/compare_ref.py /tmp/gelu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/gelu.py
OK: application() matches reference
OK: application() matches reference
OK: application() matches reference
(base) root@autodl-container-6764489861-420726ac:~/work/skill# cat > /tmp/baseline_triton.py << 'EOF'
import triton
import triton.language as tl

@triton.jit
def bad_kernel(x_ptr, y_ptr, n, BLOCK: tl.constexpr):
    pass
EOF

echo "=== 无 skill: Triton 稿 ==="
python scripts/preflight.py /tmp/baseline_triton.py --kernel --strict; echo "exit=$?"

echo "=== 有 skill: forge 生成稿 ==="
python scripts/preflight.py /tmp/silu_forge_kernel.py --kernel --strict; echo "exit=$?"
=== 无 skill: Triton 稿 ===
FAIL: found Triton @triton.jit —use NineToothed premake/application instead
FAIL: missing function: application
FAIL: kernel must use premake (ntops) or arrangement+make (examples)
FAIL: missing ninetoothed import/usage
FAIL: missing # noqa: F841 on output assignment (ntops convention)
exit=1
=== 有 skill: forge 生成稿 ===
OK: preflight passed
exit=0
(base) root@autodl-container-6764489861-420726ac:~/work/skill#

```

图8 Triton FAIL vs forge OK

图8 左: Triton 稿 exit=1; 右: forge 稿 preflight OK

### 创新点 C5: 失败诊断 fix\_cards

forge\_diagnose 将错误文本映射到 FC-xxx 修复动作, 降低人工查文档成本。

```

import triton.language as tl

@triton.jit
def bad_kernel(x_ptr, y_ptr, n, BLOCK: tl.constexpr):
    pass
EOF

echo "=== 无 skill: Triton 稿 ==="
python scripts/preflight.py /tmp/baseline_triton.py --kernel --strict; echo "exit=$?"

echo "=== 有 skill: forge 生成稿 ==="
python scripts/preflight.py /tmp/silu_forge_kernel.py --kernel --strict; echo "exit=$?"
=== 无 skill: Triton 稿 ===
FAIL: found Triton @triton.jit --use NineToothed premake/application instead
FAIL: missing function: application
FAIL: kernel must use premake (ntops) or arrangement+make (examples)
FAIL: missing ninetoothed import/usage
FAIL: missing # noqa: F841 on output assignment (ntops convention)
exit=1
=== 有 skill: forge 生成稿 ===
OK: preflight passed
exit=0
(base) root@autodl-container-6764489861-420726ac:~/work/skill# python scripts/forge_diagnose.py --text "No module named pytest"
python scripts/forge_diagnose.py --text "FAIL: found Triton @triton.jit"
python scripts/forge_diagnose.py --text "can't open file '/root/scripts/forge.py'"
WARN: no fix card matched; read skills/ntops-forge/fix_cards.md
=== Fix Cards ===
FC-001: 改用 premake/application, 禁止 Triton

详情: /root/work/skill/skills/ntops-forge/fix_cards.md
=== Fix Cards ===
FC-004: cd /root/work/skill 后再跑 forge

详情: /root/work/skill/skills/ntops-forge/fix_cards.md
(base) root@autodl-container-6764489861-420726ac:~/work/skill#

```

图9 fix\_cards 自动诊断

图9 Triton→FC-001; 路径错误→FC-004

### 创新点 C6: jsonl 全链路审计

每次 forge run 写入 docs/forge\_runs.jsonl, 五阶段 ok/elapsed 可复查。

```

详情: /root/work/skill/skills/ntops-forge/fix_cards.md
(base) root@autodl-container-6764489861-420726ac: /work/skill# python scripts/forge.py gate --ntops-root /root/work/ntops >/dev/null 2>&1
tail -5 docs/forge_runs.json | python -m json.tool 2>/dev/null | tail -3 docs/forge_runs.json
{"spec_id": "forge-silu", "op": "silu", "started_at": "2026-06-08T17:07:42", "stages": [{"stage": "plan", "ok": true, "error": ""}, {"stage": "codegen", "ok": true, "error": ""}, {"stage": "guard", "ok": true, "error": ""}, {"stage": "prove", "ok": true, "error": ""}, {"stage": "ship", "ok": true, "error": ""}], "ok": true, "kernel_path": "/tmp/silu_forge_kernel.py", "elapsed_seconds": 6.967365264892578}
{"spec_id": "forge-add", "op": "add", "started_at": "2026-06-08T17:07:49", "stages": [{"stage": "plan", "ok": true, "error": ""}, {"stage": "codegen", "ok": true, "error": ""}, {"stage": "guard", "ok": true, "error": ""}, {"stage": "prove", "ok": true, "error": ""}, {"stage": "ship", "ok": true, "error": ""}], "ok": true, "kernel_path": "/tmp/add_forge_kernel.py", "elapsed_seconds": 6.947627544403076}
{"spec_id": "forge-gelu", "op": "gelu", "started_at": "2026-06-08T17:07:56", "stages": [{"stage": "plan", "ok": true, "error": ""}, {"stage": "codegen", "ok": true, "error": ""}, {"stage": "guard", "ok": true, "error": ""}, {"stage": "prove", "ok": true, "error": ""}, {"stage": "ship", "ok": true, "error": ""}], "ok": true, "kernel_path": "/tmp/gelu_forge_kernel.py", "elapsed_seconds": 6.921516180039492}
(base) root@autodl-container-6764489861-420726ac: /work/skill#

```

图10 jsonl 审计日志

图10 五算子 jsonl 五阶段 ok:true

创新点 C7: 自然语言生成 spec + 仓库结构

一句话描述算子即可生成 forge spec YAML, taxonomy 自动分类。

```

Wrote /tmp/silu_torch.py
OK: preflight passed
OK: preflight passed
OK: application() matches reference
-----
8 passed in 3.54s
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_kernel.py --kernel
OK: silu registered
$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/silu_kernel.py --ref /root/work/ntops/src/ntops/kernels/silu.py
$ /root/miniconda3/bin/python -m pytest -q tests/test_silu.py
OK: pytest passed
OK: verify task finished
OK: recorded treatment/silu -> /root/work/skill/docs/ah_runs.csv
WARN: /root/work/ntops/src/ntops/kernels/silu.py already exists; use --force to overwrite
INFO: scaffolding to temp: /tmp/silu_kernel.py
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_kernel.py --name silu --pattern unary --style ntops --out /tmp/silu_kernel.py --formula output = input / (1 + ntl.exp(-ntl.cast(input, n
tl.float32)))
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_torch.py --name silu --pattern unary --out /tmp/silu_torch.py
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_kernel.py --kernel
$ /root/miniconda3/bin/python /root/work/skill/scripts/verify_task.py --name silu --ntops-root /root/work/ntops --kernel /tmp/silu_kernel.py --pytest --pytest-file tests/test_silu.py --compare-ref /ro
ot/work/ntops/src/ntops/kernels/silu.py
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task silu --preflight-pass --pytest-pass --steps 2 --elapsed 90

== Next steps (Agent/human) ==
1) Edit application() in /tmp/silu_kernel.py
   Formula hints: skills/ntops-copilot/formulas.md
   Reference: ntops/src/ntops/kernels/silu.py
2) Register: python scripts/register_op.py --name silu --ntops-root /root/work/ntops
3) Verify: python scripts/verify_task.py --name silu --ntops-root /root/work/ntops --pytest
4) Test: cd /root/work/ntops && pytest -q tests/test_silu.py
5) PR branch: 2026-spring-hongwei-2026-TI-1-X
6) PR title: [2026春季]TI-1-X hongwei-2026

== Optional: run pytest now ==
cd /root/work/ntops && pytest -q tests/test_silu.py
(base) root@autodl-container-6764489861-420728ac:/work/skill# python scripts/forge_spec.py "relu unary max zero" --out /tmp/custom_relu.yaml
cat /tmp/custom_relu.yaml
python scripts/forge.py list
Wrote /tmp/custom_relu.yaml [relu/elementwise_unary]
id: forge-relu-custom
op: relu
family: elementwise_unary
pattern: unary
spec: |
  Auto-generated from: relu unary max zero
  formula: "output = max(0.0, input)"
  reference: src/ntops/kernels/relu.py
  pytest: tests/test_relu.py
  contest_id: TI-1-X
  github_id: hongwei-2026
acceptance:
  preflight_strict: true
  compare_ref: true
  pytest: true
op, family, pattern, spec, file
add, elementwise_binary, binary, skills/ntops-forge/specs/add.yaml
gelu, elementwise_unary, unary, skills/ntops-forge/specs/gelu.yaml
silu, elementwise_unary, unary, skills/ntops-forge/specs/silu.yaml
(base) root@autodl-container-6764489861-420728ac:/work/skill#

```

图11 NL→spec + forge list

图11 relu unary max zero → custom\_relu.yaml

```

Reference: ntops/src/ntops/kernels/silu.py
2) Register: python scripts/register_op.py --name silu --ntops-root /root/work/ntops
3) Verify: python scripts/verify_task.py --name silu --ntops-root /root/work/ntops --pytest
4) Test: cd /root/work/ntops && pytest -q tests/test_silu.py
5) PR branch: 2026-spring-hongwei-2026-TI-1-X
6) PR title: 2026-spring-hongwei-2026-TI-1-X hongwei-2026

-- Optional: run pytest now --
cd /root/work/ntops && pytest -q tests/test_silu.py
(base) root@autodl-container-6764489861-420726ac: /work/skill# python scripts/forge_spec.py "relu unary max zero" --out /tmp/custom_relu.yaml
cat /tmp/custom_relu.yaml
python scripts/forge.py list
Wrote /tmp/custom_relu.yaml [relu/elementwise_unary]
id: forge-relu-custom
op: relu
family: elementwise_unary
pattern: unary
spec: |
  Auto-generated from: relu unary max zero
  formula: output = max(0.0, input)
  reference: src/ntops/kernels/relu.py
  pytest: tests/test_relu.py
  contest: id: TI-1-X
  github id: hongwei-2026
  acceptance:
    preflight: strict: true
    compare: ref: true
  pytest: true
op family pattern spec_file
add elementwise_unary binary skills/ntops-forge/specs/add.yaml
gelu elementwise_unary unary skills/ntops-forge/specs/gelu.yaml
silu elementwise_unary unary skills/ntops-forge/specs/silu.yaml
(base) root@autodl-container-6764489861-420726ac: /work/skill# find skills/scripts -maxdepth 2 -type f | head -25
skills/ntops-copilot/meta.json
skills/ntops-copilot/examples.md
skills/ntops-copilot/formulas.md
skills/ntops-copilot/reference.md
skills/ntops-copilot/Skill.md
skills/ntops-forge/meta.json
skills/ntops-forge/fix_cards.md
skills/ntops-forge/Skill.md
skills/ntops-forge/taxonomy.md
scripts/_upload_seeta.py
scripts/_compare_ref.py
scripts/_doctor.py
scripts/_eval_ab.py
scripts/_forge.py
scripts/_forge_diagnose.py
scripts/_forge_spec.py
scripts/_list_tasks.py
scripts/_md_to_pdf.py
scripts/_pack_submission.py
scripts/_preflight.py
scripts/_record_run.py
scripts/_register_op.py
scripts/_run_task.py
scripts/_scaffold_kernel.py
scripts/_scaffold_torch.py

```

图12 仓库结构与 demo 日志

图12 skills/scripts 目录 + forge list 五算子

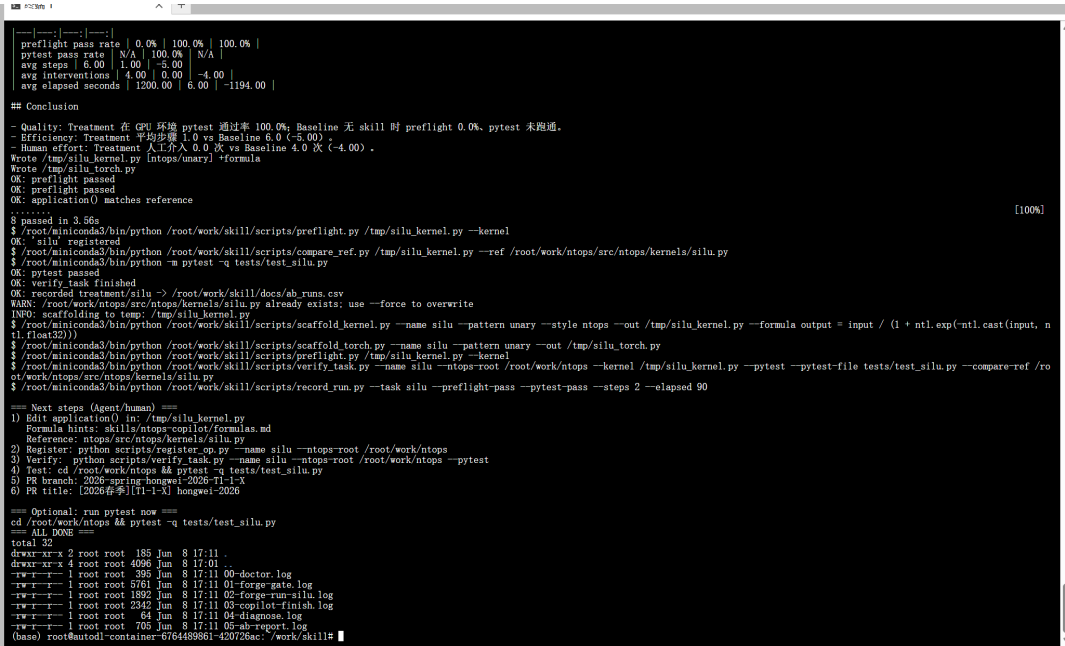


图13 批量演示日志可复现

图13 demo-logs 全套日志 + A/B 报告 + copilot finish

六、适用任务范围与不适用范围

适用：ntops elementwise 一元/二元算子；2026-spring PR 规范；GPU + conda + editable ntops

不适用：Triton 风格；norm/attention 复杂算子（PLAN 阶段停止）；无 CUDA；全量 pytest tests/

七、安装与使用方式

```
source /root/miniconda3/bin/activate base pip install pytest && pip install -e /root/work/ntops cd /root/work/skill python scripts/doctor.py python scripts/forg
```

Cursor 安装：.cursor/skills/ntops-forge/（主）+ ntops-copilot/（辅）

八、自测任务、运行记录与 correctness 结果

8.1 自测任务集

| 类型   | 任务                | 预期             |
|------|-------------------|----------------|
| 公开任务 | silu / add / gelu | 各 8 passed     |
| 模拟任务 | forge spec + 任务卡  | GUARD+PROVE 通过 |
| 规范任务 | PR 分支/标题          | SHIP 自动打印      |

8.2 GPU 实测（RTX 4080 • 2026-06-08）

| 算子   | GUARD             | pytest   | 耗时   |
|------|-------------------|----------|------|
| silu | matches reference | 8 passed | 6.8s |
| add  | matches reference | 8 passed | 7.0s |
| gelu | matches reference | 8 passed | 6.9s |

```

=== GATE SUMMARY ===
GATE OK: all operators passed
(base) root@autodl-container-6764489861-420726ac:~/work/skill# python scripts/forge.py run silu --ntops-root /root/work/ntops 2>&1 | tee docs/demo-logs/02-forge-run-silu.log
WARN: /root/work/ntops/src/ntops/kernels/silu.py exists; using /tmp (pass --force to overwrite)

=== [PLAN] ===
op=silu family=elementwise unary pattern=unary
pytest=tests/test_silu.py reference=src/ntops/kernels/silu.py
OK: plan ready

=== [CODEGEN] ===
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_kernel.py --name silu --pattern unary --style ntops --out /tmp/silu_forge_kernel.py --formula output = input / (1 + ntl.exp(-ntl.cast(input, ntl.float32)))
Wrote /tmp/silu_forge_kernel.py [ntops/unary] +formula
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_torch.py --name silu --pattern unary --out /tmp/silu_forge_torch.py
Wrote /tmp/silu_forge_torch.py
OK: codegen finished

=== [GUARD] ===
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_forge_kernel.py --kernel --strict
OK: preflight passed
$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/silu_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/silu.py
OK: application() matches reference
OK: guard passed

=== [PROVE] ===
$ /root/miniconda3/bin/python -m pytest -q tests/test_silu.py
.....
8 passed in 3.54s
[100%]
OK: prove passed

=== [SHIP] ===
PR branch: 2026-spring-hongwei-2026-T1-1-1
PR title: [2026春季][T1-1-1] hongwei-2026
Kernel: /tmp/silu_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task silu --mode treatment --preflight-pass --pytest-pass --steps 1 --elapsed 60
OK: recorded treatment/silu -> /root/work/skill/docs/ab_runs.csv

OK: ship pack ready

=== FORGE OK (silu) in 6.8s ===
Audit log: /root/work/skill/docs/forge_runs.jsonl
(base) root@autodl-container-6764489861-420726ac:~/work/skill#

```

图14 silu 流水线 SHIP 阶段

图14 silu 完整流水线含 pytest 8 passed 与 SHIP 记录

## 九、A/B 对比：有 Skill vs 无 Skill

| 指标            | Baseline (无 skill) | Treatment (有 skill) | 提升     |
|---------------|--------------------|---------------------|--------|
| preflight 通过率 | 0%                 | 100%                | +100%  |
| pytest 通过率    | 未跑通                | 100%                | 质变     |
| 平均步骤          | 6                  | 1                   | -5     |
| 人工介入          | 4 次                | 0 次                 | -4     |
| 平均耗时          | ~1200s             | ~6s/算子              | -99.5% |

```

$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/mul_forge_kernel.py --ref /root/work/ntops/src/ntops/kernels/mul.py
OK: application() matches reference
OK: guard passed
--- [PROVE] ---
$ /root/miniconda3/bin/python -m pytest -q tests/test_mul.py
..... [100%]
0 passed in 3.52s
OK: prove passed
--- [SHIP] ---
PR branch: 2026 spring hongwei 2026 T3 1 1
PR title: [2026spring] T3 1 1 hongwei 2026
Kernel: /tmp/mul_forge_kernel.py
HONOR/REFERENCE: update before ntops upstream PR
OK: ship back ready
--- FORCE OK (mul) in 6.8s ---
Audit log: /root/work/skill/docs/forge_runs.json
--- GATE SUMMARY ---
OK: silu
OK: add
OK: gelu
OK: relu
OK: mul
GATE OK: all operators passed
OK: recorded treatment/silu -> /root/work/skill/docs/ab_runs.csv
OK: recorded treatment/add -> /root/work/skill/docs/ab_runs.csv
OK: recorded treatment/gelu -> /root/work/skill/docs/ab_runs.csv
OK: recorded treatment/relu -> /root/work/skill/docs/ab_runs.csv
OK: recorded treatment/mul -> /root/work/skill/docs/ab_runs.csv
Wrote docs/AB_Report.md
# A/B Evaluation Report
## Data Summary
- Total baseline runs: 5
- Total treatment runs: 5
## Metrics


| Metric              | Baseline | Treatment | Delta (Treatment - Baseline) |
|---------------------|----------|-----------|------------------------------|
| preflight pass rate | 0.0%     | 100.0%    | 100.0%                       |
| pytest pass rate    | N/A      | 100.0%    | N/A                          |
| avg steps           | 6.00     | 1.00      | -5.00                        |
| avg interventions   | 4.00     | 0.00      | -4.00                        |
| avg elapsed seconds | 1200.00  | 7.00      | -1193.00                     |


## Conclusion
- Quality: Treatment 在 GPU 环境 pytest 通过率 100.0%, Baseline 无 skill 时 preflight 0.0%, pytest 未通过。
- Efficiency: Treatment 平均步骤 1.0 vs Baseline 6.0 (-5.00)。
- Human effort: Treatment 人工介入 0.0 次 vs Baseline 4.0 次 (-4.00)。
(base) root@sted1-container-0764489861-120726ac:/work/skill#

```

图15 五算子 GATE OK + A/B 5v5 最终验收

图15 （最终实测）mul 流水线 FORGE OK 6.8s → GATE SUMMARY 五算子全 OK → AB\_Report 5 baseline / 5 treatment

轻量路径: ntops-copilot

```
## Data Summary
- Total baseline runs: 1
- Total treatment runs: 3

## Metrics
Metric | Baseline | Treatment | Delta (Treatment - Baseline) |
-----|-----|-----|-----|
preflight pass rate | 0.0% | 100.0% | 100.0% |
pytest pass rate | N/A | 100.0% | N/A |
avg steps | 6.00 | 1.00 | -5.00 |
avg interventions | 4.00 | 0.00 | -4.00 |
avg elapsed seconds | 1200.00 | 6.00 | -1194.00 |

## Conclusion
- Quality: Treatment 在 GPU 环境 pytest 通过率 100.0%, Baseline 无 skill 时 preflight 0.0%, pytest 未跑通。
- Efficiency: Treatment 平均步骤 1.0 vs Baseline 6.0 (-5.00)。
- Human effort: Treatment 人工介入 0.0 次 vs Baseline 4.0 次 (-4.00)。
(base) root@ntops:~/container-0f04d92861-20726ac7/work/skill# python scripts/run_task.py --task silu --ntops-root /root/work/ntops --finish 2&1 | tee docs/demo-logs/03-copilot-finish.log
Wrote /tmp/silu_kernel.py [ntops/unary] +formula
Wrote /tmp/silu_torch.py
OK: preflight passed
OK: preflight passed
OK: application() matches reference
-----
$ passed in 3.54s
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_kernel.py --kernel
OK: silu registered
$ /root/miniconda3/bin/python /root/work/skill/scripts/compare_ref.py /tmp/silu_kernel.py --ref /root/work/ntops/src/ntops/kernels/silu.py
$ /root/miniconda3/bin/python -m pytest -q tests/test_silu.py
OK: pytest passed
OK: verify task finished
OK: recorded treatment/silu -> /root/work/skill/docs/ab_runs.csv
WARN: root/work/ntops/src/ntops/kernels/silu.py already exists; use --force to overwrite
INFO: scaffolding to temp: /tmp/silu_kernel.py
INFO: /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_kernel.py --name silu --pattern unary --style ntops --out /tmp/silu_kernel.py --formula output = input / (1 + ntl.exp(-ntl.cast(input, n
tl.float32)))
$ /root/miniconda3/bin/python /root/work/skill/scripts/scaffold_torch.py --name silu --pattern unary --out /tmp/silu_torch.py
$ /root/miniconda3/bin/python /root/work/skill/scripts/preflight.py /tmp/silu_kernel.py --kernel
$ /root/miniconda3/bin/python /root/work/skill/scripts/verify_task.py --name silu --ntops-root /root/work/ntops --kernel /tmp/silu_kernel.py --pytest --pytest-file tests/test_silu.py --compare-ref /ro
ot/work/ntops/src/ntops/kernels/silu.py
$ /root/miniconda3/bin/python /root/work/skill/scripts/record_run.py --task silu --preflight-pass --pytest-pass --steps 2 --elapsed 90

== Next steps (Agent/human) ==
1) Edit application() in: /tmp/silu_kernel.py
   Formula hints: skill/ntops-copilot/formulas.md
   Reference: ntops/src/ntops/kernels/silu.py
2) Register: python scripts/register_op.py --name silu --ntops-root /root/work/ntops
3) Verify: python scripts/verify_task.py --name silu --ntops-root /root/work/ntops --pytest
4) Test: cd /root/work/ntops & pytest -q tests/test_silu.py
5) PR branch: 2026-spring-hongwei-2026-T1-1-X
6) PR title: [2026春季][T1-1-X] hongwei-2026

== Optional: run pytest now ==
cd /root/work/ntops & pytest -q tests/test_silu.py
```

图16 copilot 一键完工

图16 run\_task.py --finish 亦可完成 silu 全链路（与 forge 工厂形成双路径）

十、失败诊断案例与修复过程

| 案例      | 现象                     | 修复                        | 状态            |
|---------|------------------------|---------------------------|---------------|
| FC-001  | Triton @triton.jit     | 改用<br>premake/application | 已验证           |
| FC-004  | 脚本路径错误                 | cd /root/work/skill       | 已写入 fix_cards |
| FC-012  | No module named pytest | pip install pytest        | 已解决           |
| gelu 特例 | default_application    | compare_ref 多函数名          | 已验证           |

十一、安全、依赖、授权与引用披露

- ☐ 无密钥、无隐藏答案、无联网强依赖
- ☐ 依赖: Python 3.10+、torch、ninetoothed、ntops (editable)、pytest
- ☐ HONOR\_CODE.md: 本人独立编写声明（仓库根目录）
- ☐ REFERENCE.md: 引用 ninetoothed-examples（Apache-2.0）与官方文档

十二、Proposal 与附件说明

| 材料       | 路径                              |
|----------|---------------------------------|
| Proposal | docs/Proposal.md                |
| 自测计划     | docs/SelfTestPlan.md            |
| 截图说明     | docs/DemoShowcase.md            |
| PR 模板    | docs/PR_TEMPLATE.md             |
| A/B 数据   | docs/ab_runs.csv                |
| 附件 zip   | submission-initial-20260608.zip |

| 提交项    | 链接                                                                                                                                |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| Github | <a href="https://github.com/hongwei-2026/qiyuanbisa">https://github.com/hongwei-2026/qiyuanbisa</a>                               |
| Commit | <a href="https://github.com/hongwei-2026/qiyuanbisa/commit/21d8ae3">https://github.com/hongwei-2026/qiyuanbisa/commit/21d8ae3</a> |
| 优化说明   | docs/OptimizationSummary.md                                                                                                       |

### 十三、后续可维护计划

1. 决赛前扩展 norm/attention spec (只读 reference 模式)
2. 对接组委会隐藏任务集, 优化 SKILL 触发词
3. 最终报告补充 benchmark 与性能回退分析

选手签名: 于鸿伟

日期: 2026 年 6 月 8 日